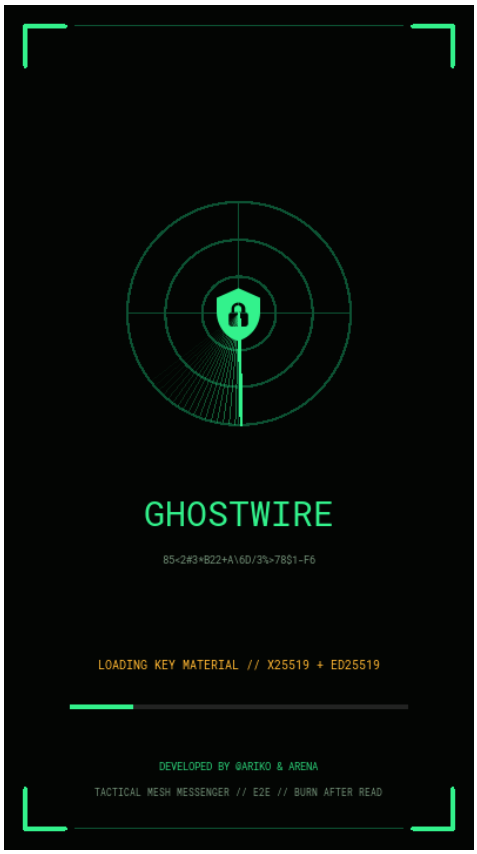
GHOSTWIRE

Operator's Manual · v1.1.0
Tactical encrypted messenger // GW1 protocol



Install it. Deploy a node. Verify your peer. Chat.
Everything in this book is illustrated with real screenshots from the app.

Contents

1 Welcome, operative 3

2 Installing and first launch 3

3 Home screen and your identity 4

4 Deploying your node (receiving side) 5

5 Finding peers — network sweep 6

6 Calling out — establish uplink 7

7 The request gate — acknowledge or reject 9

8 Chatting on the secure channel 10

9 Verifying fingerprints (do this once per session) 12

10 Burning and disconnecting 13

11 Security Protocols screen 14

12 Stealth mode 15

13 Chatting across the internet — the WAN relay 16

14 Troubleshooting 17

15 Field habits — the one-page doctrine 18

1 Welcome, operative

GHOSTWIRE is a tactical encrypted messenger for two people who need to talk privately on a network they do not control. There are no accounts, no servers holding your messages, and no chat history: two devices form a one-to-one encrypted channel, exchange messages that self-destruct on timers you choose, and leave nothing behind.

1.1 What you get in v1.1.0

- Strictly peer-to-peer. One node, one live link, two parties. Every incoming connection waits at a request gate until you personally ACKNOWLEDGE or REJECT it — nobody can silently join or knock your peer off.
- Military-grade encryption, automatic. X25519 key exchange, Ed25519 mutual signatures and AES-256-GCM message encryption with keys that rotate on every single message. You never touch a key.
- Burn-after-reading timers. Each message carries a fuse: 60 seconds by default, or 1.5, 2, 3 and 5 minutes. Both sides delete the message when the fuse hits zero.
- Three transports. Wi-Fi/LAN, Bluetooth (RFCOMM), and a WAN relay for chatting across the internet through a relay that is blind by design — it can neither read nor forge your traffic.
- Stealth mode. Your node can go silent to network sweeps while still accepting connections from peers who know your address.
- Radical transparency. A built-in SECURITY PROTOCOLS screen names every cipher and standard in force, so anyone can audit what the app claims.

1.2 What you need

- Two Android devices running Android 10 or newer, or one Android device plus a desktop for testing.
- The install file ghostwire-1.1.0-arm64-v8a-debug.apk.
- A shared network (both on the same Wi-Fi), Bluetooth range — or the relay script on any computer both devices can reach (Chapter 9).
- A second channel for one short job: comparing fingerprint codes by voice or in person (Chapter 7).

> No accounts, no cloud — GHOSTWIRE has no login, no contacts server and no message store. Your identity is a key created on first launch and kept only on your device. If you uninstall, that identity is gone — which is exactly the point.

2 Installing and first launch

2.1 Install the APK

- Copy ghostwire-1.1.0-arm64-v8a-debug.apk to the phone (USB, Bluetooth file share, SD card, or a download link).
- Tap the file. If Android asks, allow Install unknown apps for the Files/Chrome app you are installing from — this is the normal sideload prompt, not a warning about GHOSTWIRE.
- Confirm Install. Because the app targets a modern Android release, you will not see the “built for an older version of Android” warning.

- Open GHOSTWIRE. Grant nothing yet — the app asks for permissions only when a feature needs them (network state always; Bluetooth only for the Bluetooth transport and only on Android 12+).

2.2 First boot

The app opens on an animated splash — radar sweep, live key-status line and the credits footer. Nothing is transmitted at boot: your identity keys are generated locally in those seconds and no network socket opens until you act. Tap anywhere to skip, or let it finish into the home screen.

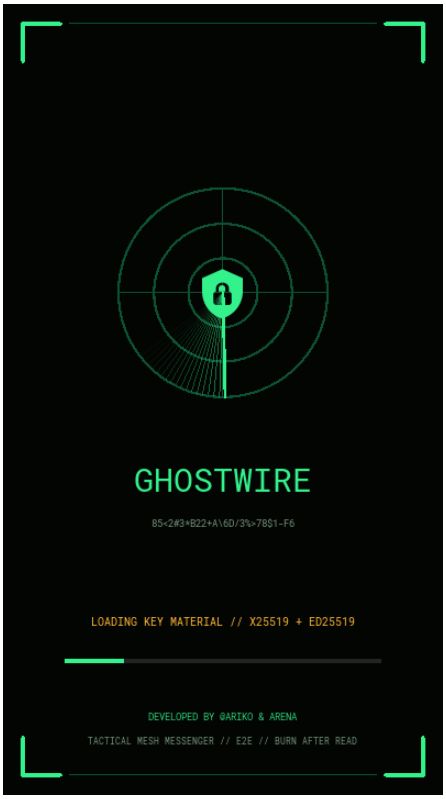


Figure 1 — First boot. Keys are generated locally; the network stays silent until you deploy a node or dial out.

3 Home screen and your identity

Home is mission control. The top card is NODE CONTROL (your listening post — Chapter 4). Below it the SITREP panel reports live state, and three buttons lead to the uplink, sweep and security screens.

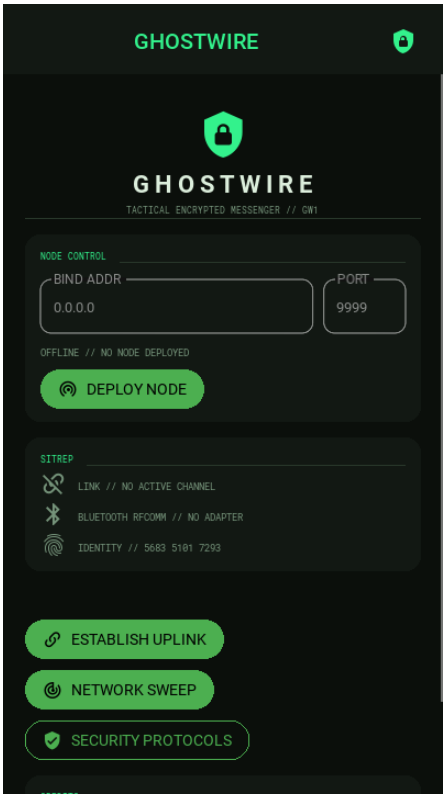


Figure 2 — Home screen with the node offline. SITREP shows link state, Bluetooth availability and your identity fingerprint.

3.1 Reading the SITREP panel

Row	Meaning
LINK	NO ACTIVE CHANNEL, or the live link plus the peer's fingerprint prefix.
BLUETOOTH RFCOMM	Adapter state on this device — needed only for the Bluetooth transport.
IDENTITY	Your permanent-on-this-install fingerprint. Peers use it to confirm they reached the real you (Chapter 7).

3.2 Your identity

The fingerprint (four groups, e.g. 5083 5101 7293 …) is the public face of a key created on first launch. It never leaves your device except inside encrypted handshakes and probe replies — and never while stealth is on. Reinstalling the app creates a fresh identity; tell your peer so they expect a new fingerprint.

> Write it down — Before your first real conversation, read your fingerprint to your peer over a different channel (a call, in person). When the same code later appears on their verification screen, the man-in-the-middle is defeated.

4 Deploying your node (receiving side)

One of the two devices must listen before the other can call. That listener is your node. Fill in the two fields and tap DEPLOY NODE:

- BIND ADDR — which network interface to listen on. 0.0.0.0 means every interface (Wi-Fi and mobile data alike); use your Wi-Fi address (e.g. 192.168.1.20) to expose the node on one network

only.

- PORT — the TCP port, 9999 unless you have a reason to move it. Your peer must dial the same port.

The status line turns green — LISTENING — and the button becomes STAND DOWN NODE. Your node now answers reconnaissance probes with its fingerprint (unless stealth is on) and parks every real caller at your request gate (Chapter 6). It never calls anyone by itself.



Figure 3 — Node deployed and listening. The SITREP still shows no active channel — listening alone creates no connection.

4.1 Standing down

Tap STAND DOWN NODE to close the listener. Any live call is dropped and its keys discarded. Good fieldcraft: deploy only for the contact window, then stow — a silent phone cannot be probed.

> Two phones, two roles — Only one side deploys a node per conversation. If both deploy, each can still dial the other — the first completed call wins. Over Bluetooth there is no node step at all: pairing happens entirely from the uplink screen.

5 Finding peers — network sweep

If both devices share a Wi-Fi network but you do not know your peer's address, sweep for it. Open NETWORK SWEEP, enter the range and port, and tap INITIATE SWEEP:

- TARGET RANGE accepts CIDR (192.168.1.0/24) or a plain list (192.168.1.7, 192.168.1.20). Sweeps are capped at 1,024 hosts.
- PORT must match the node you are hunting (9999 default).
- ABORT stops a running sweep at any time.

Each confirmed GHOSTWIRE node answers the probe with its identity fingerprint, and the sweep lists it as a tappable contact showing address + fingerprint. Tap the contact and the app carries the

address straight into the uplink screen, ready to dial.

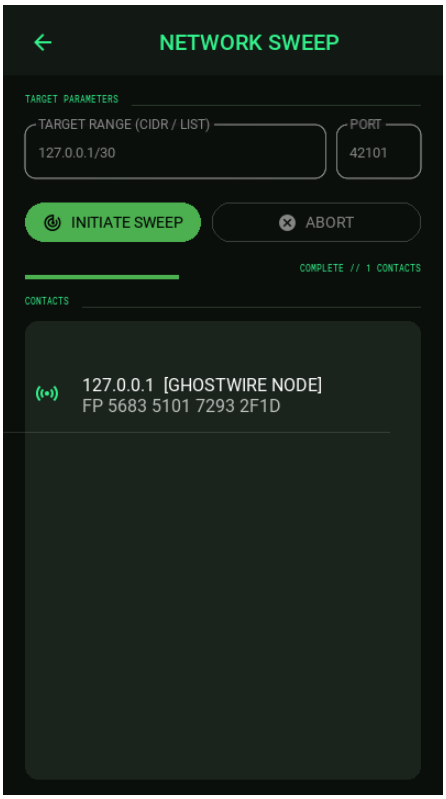


Figure 4 — A finished sweep: one contact found. Tap it to designate it as your call target.

5.1 What sweeps do — and do not — show

- Only hosts that answer with a valid GW1 fingerprint are listed. Bare open ports are ignored, so stealth nodes (Chapter 8) stay invisible — by design, not by accident.
- Absence of a result proves nothing: the peer may be stealthed, firewalled, on another port, or simply not deployed yet. When in doubt, type the address directly.
- Sweeps are meaningless across the internet — for that, exchange the relay address out-of-band instead (Chapter 9).

6 Calling out — establish uplink

Open ESTABLISH UPLINK, check the target (or tap a sweep contact to fill it in), pick a transport, and tap INITIATE HANDSHAKE. The console narrates every phase of the cryptographic handshake; success ends in SECURE LINK ESTABLISHED with a session id and your peer's fingerprint, and the app jumps to the chat screen.

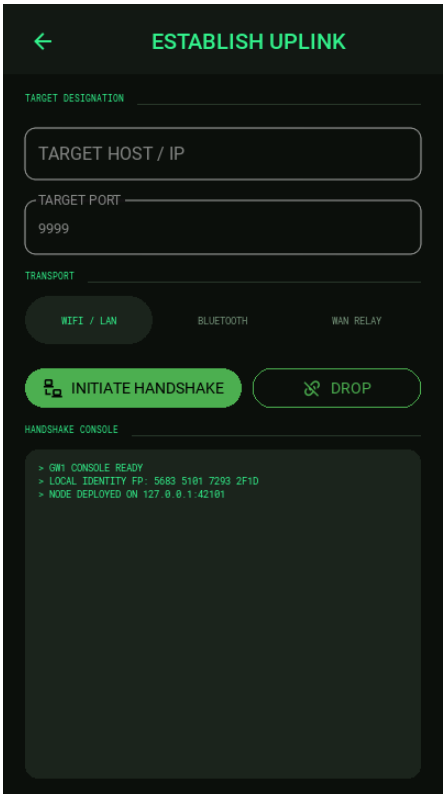


Figure 5 — Uplink screen on the Wi-Fi/LAN transport, ready to dial.

6.1 The three transports

Transport	Target field means	When to use it
WIFI / LAN	Hostname/IP + TCP port of the peer's node.	Same network, or any routable address (VPN, public IP, port forward).
BLUETOOTH	Peer's Bluetooth address + RFCOMM channel (7).	No network at all — same-room, zero infrastructure.
WAN RELAY	Relay host + port both sides dial.	Different networks / across the internet (Chapter 9).

6.2 Reading the handshake console

The console is your flight recorder. A healthy call reads top to bottom like this: channel opening, ephemeral key generation, hello exchange, signature verification, key derivation, key confirmation — then SECURE LINK ESTABLISHED. If anything fails you get the exact reason (e.g. identity signature check failed) instead of a vague spinner — treat crypto errors as stop signs, never as retry hints.

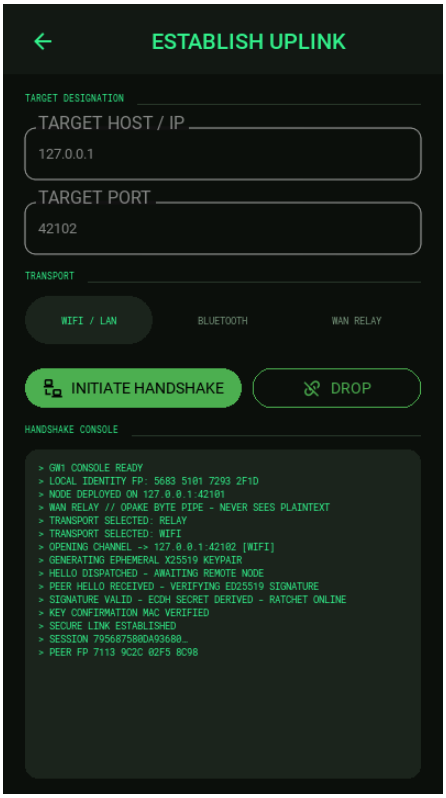


Figure 6 — A completed handshake: every phase logged, session id issued, peer fingerprint shown.

6.3 The DROP button

DROP tears the link down and discards its keys. While a link is up, the app refuses a second call (“ALREADY LINKED // DROP FIRST”) — v1.1.0 is one conversation at a time, enforced in both directions.

7 The request gate — acknowledge or reject

Here is the heart of v1.1.0. When your node is deployed and someone dials you, no encryption happens yet. The caller is parked and your screen raises an INBOUND REQUEST naming them:

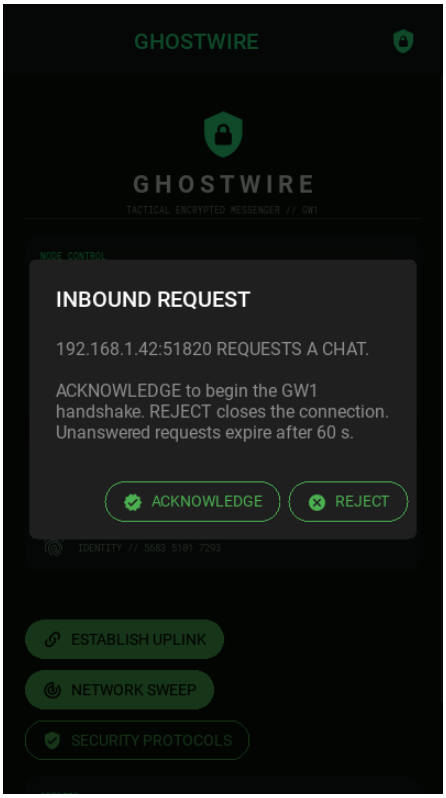


Figure 7 — The request gate. Nothing cryptographic has happened — the decision is entirely yours.

- ACKNOWLEDGE — begin the GW1 handshake with this caller. If you already have a peer connected, they are dropped first, explicitly and visibly — never silently.
- REJECT — close the socket immediately. The caller learns nothing except that the call did not complete.
- Ignore it — the request expires on its own after 60 seconds and the socket closes.

7.1 Why the gate exists

Without it, any third device that found your address could dial in and knock your real peer off the node. With it, every connection is a conscious human decision. Coordinate contact windows with your peer in advance (“dial me at 2100”) so a request arriving at the right moment is expected — and one at 0300 gets REJECT.

> Bluetooth callers wait here too — The Bluetooth node applies the identical gate with the caller's Bluetooth address shown. One policy everywhere: no anonymous hall passes.

8 Chatting on the secure channel

The chat header names the endpoint and your peer's fingerprint; the green banner states the guarantees in force: AES-256-GCM end-to-end, ratcheted keys, burn after read. Type in ENCRYPTED TRANSMISSION… and hit send — every message is sealed with a fresh key before it leaves the phone.

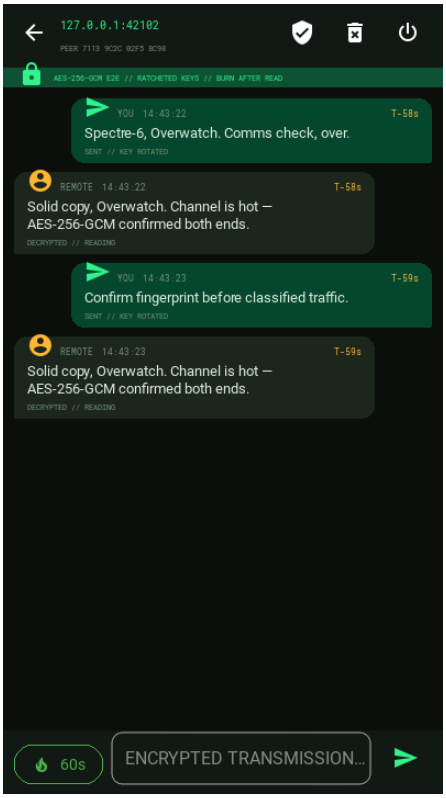


Figure 8 — A live conversation. Each bubble carries its own countdown and delivery state.

8.1 Reading a bubble

- T-58s — the fuse: seconds until this message deletes itself on both phones. It turns red under 3 seconds.
- SENT // KEY ROTATED — your message left and the next one will use new key material.
- DECRYPTED // READING — an incoming message, verified and displayed; its fuse is already running.
- READ BY PEER // KEY ROTATED — your peer's app confirmed display. INCINERATED // PEER CONFIRMED — both sides deleted it.

8.2 Message-expiry timers

The fire button at the left of the composer sets the fuse every new message carries. Tap it to cycle: 60s → 1.5m → 2m → 3m → 5m → back to 60s. Sixty seconds is the default — short enough that a seized phone yields almost nothing, long enough to actually read. Raise it for coordinates and long texts, never out of habit.

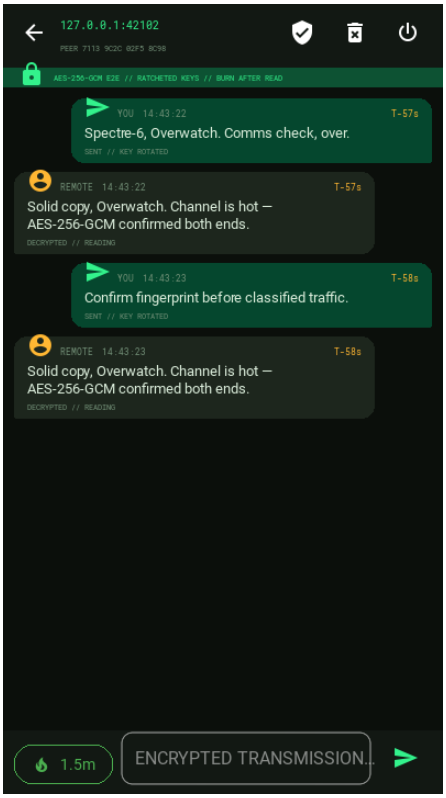


Figure 9 — Fuse set to 1.5 minutes. Every new message from here lives 90 seconds on both devices.

8.3 Header buttons

Button	Does
shield-check	Open fingerprint verification (Chapter 7's companion — do it once per session).
delete-forever	BURN: delete the whole conversation on both phones immediately.
power	Disconnect this channel. Keys die with the link.

> Messages live in memory only — Nothing is written to storage — no database, no cache, no notification log to scrub. Leaving the chat, dropping the link, or killing the app all destroy the conversation.

9 Verifying fingerprints (do this once per session)

Encryption without authentication is just expensive whispering. The verification dialog shows your peer's full fingerprint — compare it against the code you exchanged over a different channel before trusting the conversation.

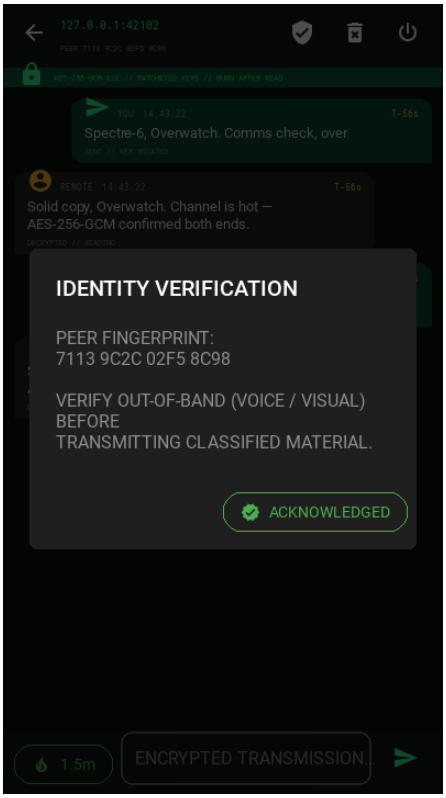


Figure 10 — Identity verification. If these groups match the code your peer read to you, no one stands between you.

- Open it any time from the chat header (shield-check).
- Read the groups aloud to each other by voice or video call — never inside the chat you are trying to verify.
- A mismatch means stop: wrong peer, stale identity after a reinstall, or active interference. Do not “retry and see”.
- Matching once per session is enough; the ratchet then vouches for every following message.

10 Burning and disconnecting

When the conversation is over — or at the first whiff of trouble — destroy it:

- delete-forever (BURN) — incinerates every message on both devices and tells your peer's app to do the same. Bubbles flash INCINERATED // PEER CONFIRMED as each deletion is acknowledged.
- power (disconnect) — drops the encrypted link. The conversation widgets are cleared and the ratchet keys are thrown away; even a memory dump taken later cannot resurrect them.
- DROP on the uplink screen — same as power, available without opening the chat.

After a disconnect the header reads LINK LOST and SITREP returns to NO ACTIVE CHANNEL. Reconnecting always runs a fresh handshake with fresh ephemeral keys — there is no “resume” that could replay an old session.

> Burn is a courtesy, not a proof — BURN asks the peer's app to delete too, and an honest app always does. But no software can stop a determined peer photographing their screen. Send accordingly: timers bound exposure, they do not bound cameras.

11 Security Protocols screen

The shield button on the home bar opens the transparency panel: everything the app claims about its own cryptography, on one scrollable screen. If any line ever disagrees with this manual, stop and investigate — the code is the spec.



Figure 11 — Cipher suite in force, live session id and peer fingerprint.

11.1 GW1 cipher suite

Primitive	Role
X25519 ECDH (ephemeral)	Each handshake derives fresh shared secrets; nothing long-term ever encrypts traffic.
Ed25519 mutual signatures	Both hellos are signed — authentication runs both directions, no passwords, no PKI.
HKDF-SHA256 + symmetric ratchet	Fresh message keys stepped per message; old state is erased (forward secrecy).
AES-256-GCM (AEAD)	Every frame encrypted and authenticated; tampering fails closed.
HMAC-SHA256 key confirm	Both sides prove identical keys before the first message may flow.
Bilateral burn	Coordinated deletion across both devices.

11.2 Standards & RFC references

Scroll down for the panel that pins each primitive to its public standard — the same inventory auditors check against:



Figure 12 — Standards panel, active session card and stealth switch.

Standard	Covers
RFC 7748	X25519 Diffie-Hellman.
RFC 8032	Ed25519 signatures.
RFC 5869	HKDF key derivation.
RFC 2104	HMAC key confirmation.
NIST SP 800-38D	AES-GCM authenticated encryption.
FIPS 180-4	SHA-256 digest.

11.3 Active session & local node cards

ACTIVE SESSION shows the live session id and peer fingerprint for read-back comparison; LOCAL NODE repeats your own fingerprint and holds the stealth switch. The green FIELD NOTE at the bottom is the developers' honesty box: confidentiality, integrity, mutual authentication, forward secrecy, transience — and the reminder that your device remains the weakest link.

12 Stealth mode

Flip the stealth switch (Security Protocols → LOCAL NODE) and your deployed node goes silent to sweeps: reconnaissance probes are swallowed without a reply, so the node never appears in anyone's contact list — including the sweep screen in this very app, which deliberately lists only confirmed GW1 answers.

- What stealth hides: your fingerprint, your presence as a GHOSTWIRE node, and any protocol banner.

- What stealth does not hide: the bare fact that something listens on that TCP port. A raw port scanner still sees an open port; move off 9999 and firewall-restrict where you can.
- What keeps working: everything else. Peers who know your address connect normally, the request gate still guards every call, and handshakes are unaffected.

Stealth pairs with pre-arranged addresses — A stealthed node cannot be discovered — only dialled. That is the intended combination: swap addresses and fingerprints over your out-of-band channel, then run silent.

13 Chatting across the internet — the WAN relay

Two devices on different networks cannot dial each other directly. For that, v1.1.0 ships relay_server.py: a tiny script you run on any computer both phones can reach (a VPS, a home server, a laptop on a public address). Both sides dial the relay with the WAN RELAY transport — and the relay introduces them without ever being able to read a word.

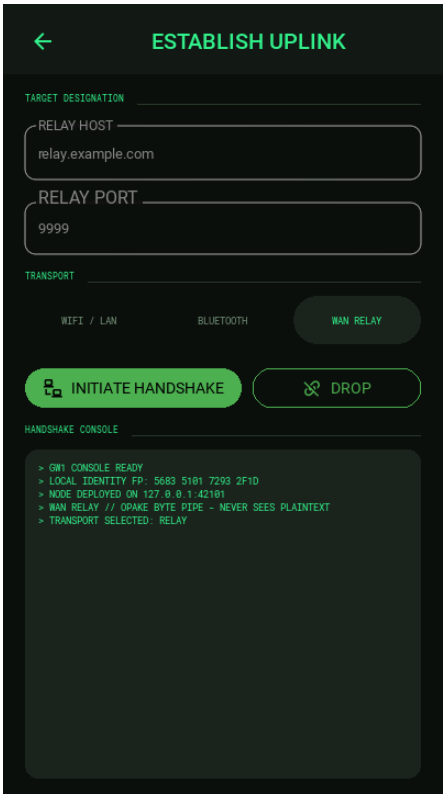


Figure 13 — Uplink screen on the WAN RELAY transport, pointed at the operator's relay.

13.1 Running the relay (5 minutes, once)

- Copy relay_server.py to the server machine. It needs only Python 3 — no packages, no root, no configuration files.
- Start it: python3 relay_server.py --host 0.0.0.0 --port 9999. Open that TCP port in the machine's firewall.
- Tell both operators the relay's address and port over your out-of-band channel, exactly like a fingerprint.
- Both phones: uplink screen → WAN RELAY → enter relay host and port → INITIATE HANDSHAKE.

13.2 What happens on dial

- The first phone to arrive is parked; the console says it is waiting for a peer (up to 2 minutes).
- When the second phone dials, the relay pairs them: first arrival takes the GW1 server seat, second the client seat.
- From there the ordinary GW1 handshake runs through the pipe — keys are derived end-to-end by the two phones. The relay shuttles bytes it cannot decrypt, then tears the pairing down when either side hangs up.

13.3 Why the relay cannot spy on you

- It performs no GW1 parsing and holds no keys — hellos, confirmations and every message cross it as opaque ciphertext.
- It sees only metadata: who dialled, when, and how many bytes moved. Run your own relay and even that stays in your hands.
- Reconnaissance probes sent to it are closed silently — it has no identity to reveal, because it has none.

```
$ python3 relay_server.py --host 0.0.0.0 --port 9999
[relay] listening on 0.0.0.0:9999 (max 16 concurrent pairs)
[relay] 197.232.61.9:51244 parked, waiting for peer
[relay] pairing 41.90.63.200:38810 with established peer
[relay] pair 41.90.63.200:38810 closed after 18411 bytes one-way
```

One relay, many pairs — A single relay hosts up to 16 simultaneous pairs by default (--max-pairs). Each pair is isolated: bytes from one conversation can never leak into another.

14 Troubleshooting

The handshake console always prints the precise failure class. Find yours below:

Symptom	Likely cause	What to do
REQUEST EXPIRED / call times out at the gate	Peer never tapped ACKNOWLEDGE within 60 s.	Coordinate the window first; then re-dial. Never loop redial.
CONNECTION REJECTED	Peer tapped REJECT — or a stranger probed you and you rejected them.	Expected behaviour. Arrange the call out-of-band.
HANDSHAKE REJECTED (SecurityError)	Tampered/replayed hello, version mismatch, role confusion.	Do not retry blindly. Verify addresses and fingerprints.
identity signature check failed	Wrong/stale peer key or substitution on the path.	Compare fingerprints; abort if they differ.
key confirmation MAC mismatch	Ends derived different keys.	Abort and treat as an attack until proven otherwise.
Nothing found in SWEEP	Stealth node, firewall, wrong port, or node not deployed.	Dial the address directly; confirm port and deploy state.

Symptom	Likely cause	What to do
ALREADY LINKED // DROP FIRST	A link is already up.	Finish or DROP the current call first — one at a time.
Relay dial waits forever	Partner never dialled, or wrong relay address/port.	Confirm both sides use the same relay; check the relay log.
Bluetooth failure	No adapter, out of range, wrong address.	Check SITREP adapter line, range (<10 m), re-enter BD_ADDR.

Crypto errors are stop signs — Anything containing SecurityError, signature, MAC or confirm means the mathematics refused — the app is protecting you. The correct response is out-of-band verification, never hammering INITIATE HANDSHAKE.

15 Field habits — the one-page doctrine

15.1 Before every real conversation

- ☐ Fingerprints exchanged over an independent channel.
- ☐ Contact window agreed (“dial me at 2100, port 9999”).
- ☐ Node deployed only for the window (or relay confirmed up).
- ☐ Stealth ON if discovery must fail.

15.2 During

- ☐ ACKNOWLEDGE only the expected caller; REJECT the rest.
- ☐ Verify fingerprints in-app against the exchanged codes.
- ☐ Keep the fuse short (60 s default); raise it per message, not per habit.
- ☐ Read the session id back verbally once; both ends match.

15.3 After

- ☐ BURN the conversation on both ends.
- ☐ Disconnect, then STAND DOWN the node.
- ☐ New install anywhere → new identity → re-verify next time.

15.4 Never

- Never verify a fingerprint inside the chat being verified.
- Never retry a crypto error without an out-of-band check.
- Never trust a relay you do not control with your metadata — run your own; it is one file.
- Never assume a timer defeats a camera. Send accordingly.

End of manual. Stay silent, stay verified — developed by @Ariko & Arena.